
Framework de desarrollo seguro basado en métodos computacionales guiados por conocimiento

Plan técnico de desarrollo, arquitectura y selección de herramientas
DevSecOps para arquitecturas de microservicios bajo el ciclo Design Science Research

Índice general

1. Introducción	3
1.1. Propósito del documento	3
1.2. Alcance	3
1.3. Criterios de selección de herramientas	3
2. Principios de diseño	4
3. Arquitectura general	5
3.1. Las tres capas del framework	5
3.2. Vista de despliegue: dónde corre cada cosa	5
4. Stack tecnológico completo	7
4.1. Resumen del stack	7
4.2. Lenguajes de programación: dónde se usa cada uno	8
4.2.1. Por qué FastAPI y no otro framework	9
4.3. Interfaz: Streamlit o Next.js	9
5. Núcleo de conocimiento (capa superior)	10
5.1. Base de conocimiento	10
5.1.1. Qué contiene	10
5.1.2. Cómo se almacena	10
5.1.3. Por qué PostgreSQL + pgvector	10
5.2. Motor de razonamiento	11
5.2.1. Por qué OPA	12
5.3. Trazabilidad y retroalimentación	12
5.4. SARIF como formato común	13
6. Uso de modelos de lenguaje (LLM)	14
6.1. Principio de uso	14
6.2. Mapa de uso por fase	14
6.3. Selección de modelos	15
6.4. Adaptador de proveedor	15
6.5. Riesgos específicos del uso de LLM	16
7. Pipeline de construcción y validación (capa media)	17
7.1. Fase 1: Adquisición de requisitos	17
7.2. Fase 2: Análisis y propuesta arquitectónica	17
7.3. Fase 3: Generación de la estructura interna	18
7.4. Fase 4: Security Gate 1, análisis estático	18
7.5. Fases 5 y 6: Refinamiento iterativo guiado por conocimiento	19
7.5.1. Salvaguardas obligatorias	20
7.6. Fase 7: Integración de microservicios	20
7.7. Fase 8: Security Gate 2, seguridad de integración	21
7.8. Fase 9: Pruebas automatizadas	21
7.9. Fase 10: Contenerización	22
7.10. Fase 11: DAST y Security Gate 3	22

8. Infraestructura y CI/CD	23
8.1. GitHub Actions como orquestador del pipeline	23
8.2. Entornos	23
9. Entradas y salidas del framework	25
9.1. Formas de entrada	25
9.2. Salida: el entregable	25
9.3. Informe de evidencia y trazabilidad	25
10. Seguridad del propio framework	26
11. Evaluación del artefacto (ciclo DSR)	27
11.1. Relación entre el ciclo DSR y el plan	27
11.2. Métricas	27
11.3. Diseño experimental	27
11.3.1. Conjuntos de requisitos	27
11.3.2. Líneas base de comparación	28
11.3.3. Calibración de las herramientas dinámicas	28
11.4. Dashboard de métricas	28
12. Estructura del repositorio del framework	29
13. Riesgos del proyecto y mitigación	30
A. Glosario	31

1. Introducción

1.1 Propósito del documento

El objetivo del proyecto es automatizar e integrar la seguridad (DevSecOps) y las buenas prácticas de diseño en todo el ciclo de vida del software para arquitecturas de microservicios, usando conocimiento formalizado y reglas computacionales en lugar de revisiones manuales aisladas.

Este documento es la guía de implementación. Para cada componente del framework indica qué herramienta se usa, para qué, por qué se eligió frente a sus alternativas y qué produce. Está pensado para que el equipo pueda seguirlo de principio a fin sin decisiones tecnológicas pendientes.

1.2 Alcance

El framework recibe un documento de requisitos en lenguaje natural y entrega un sistema de microservicios contenerizado, probado y validado en seguridad, acompañado de un informe de evidencia que traza cada requisito hasta el código y los controles que lo satisfacen.

- **Dentro del alcance:** generación de microservicios en Python/FastAPI, tres Security Gates automatizados, refinamiento iterativo guiado por conocimiento, integración con API Gateway, event bus e Identity Provider, pruebas automatizadas, análisis dinámico y trazabilidad completa.
- **Fuera del alcance (trabajo futuro):** generación en otros lenguajes, despliegue en Kubernetes, service mesh, despliegue a producción real.

1.3 Criterios de selección de herramientas

Cada herramienta se eligió aplicando los mismos cinco criterios, en este orden de prioridad:

1. **Beneficio para la pregunta de investigación:** que aporte evidencia medible al framework.
2. **Facilidad de adopción:** curva de aprendizaje baja, buena documentación, instalación con un comando o una imagen Docker.
3. **Integración en pipeline:** ejecutable por línea de comandos, salida en formato estructurado (idealmente SARIF o JSON) y tiempos de ejecución compatibles con CI.
4. **Licencia abierta y coste cero:** reproducibilidad académica sin depender de licencias comerciales.
5. **Madurez y comunidad:** proyecto activo, respaldado por una fundación u organización reconocida (OWASP, CNCF, etc.).

2. Principios de diseño

Estos principios guían todas las decisiones técnicas del documento. Cuando surja una duda durante la implementación, se resuelve volviendo a ellos.

P1. El LLM propone, las reglas deciden

Los modelos de lenguaje generan, extraen, corrigen y explican. Nunca aprueban un Security Gate ni toman una decisión arquitectónica final. Toda decisión que afecte la seguridad la toma un motor de reglas determinista (Open Policy Agent) a partir de la base de conocimiento, y queda registrada con la regla que la justifica. Así cada decisión es reproducible, auditable y explicable.

P2. Un formato común para todos los resultados

Todas las herramientas de análisis (SAST, SCA, secretos, contenedores, DAST) se normalizan al formato estándar SARIF. Esto permite comparar hallazgos entre iteraciones, detectar regresiones mediante *fingerprints* y alimentar la trazabilidad con una sola estructura de datos.

P3. Primero de punta a punta, después inteligente

La primera entrega es un *walking skeleton*: un microservicio escrito a mano que atraviesa los tres Security Gates en CI. Solo cuando la tubería funciona completa se añade la generación con LLM y el razonamiento. Esto evita descubrir problemas de integración al final del proyecto.

P4. Un solo lenguaje, bien resuelto

El motor y los microservicios generados usan Python. Cada lenguaje adicional multiplica plantillas, reglas y pruebas. La extensibilidad a otros lenguajes se diseña (plantillas y reglas intercambiables) pero no se implementa en esta versión.

P5. Conocimiento como código

La base de conocimiento vive en archivos versionados en Git (YAML, Rego y reglas Semgrep). Cada cambio en el conocimiento tiene autor, fecha y motivo, y la evaluación puede reproducir exactamente con qué versión del conocimiento se obtuvo cada resultado.

3. Arquitectura general

3.1 Las tres capas del framework

El framework se organiza en tres capas, tal como el diagrama conceptual del proyecto:

- **Capa superior, núcleo de conocimiento (KBS):** base de conocimiento, motor de razonamiento y trazabilidad. Da soporte a todas las fases.
- **Capa media, pipeline de construcción y validación:** once fases desde los requisitos hasta los contenedores validados, con tres Security Gates y un ciclo de refinamiento.
- **Capa inferior, marco metodológico:** el ciclo Design Science Research, que organiza la construcción y la evaluación del propio framework.

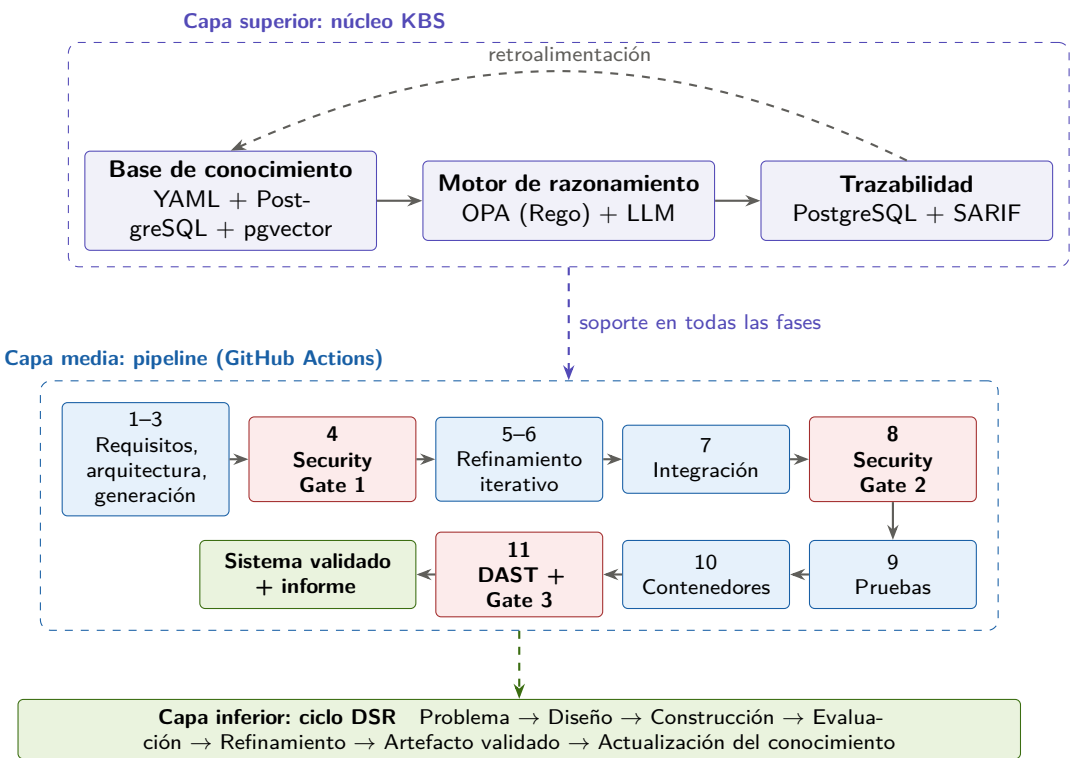


Figura 3.1: Vista general del framework y de las tecnologías de cada capa.

3.2 Vista de despliegue: dónde corre cada cosa

Una decisión crítica es dónde se ejecuta el motor. El pipeline necesita ejecutar Docker (para levantar los microservicios, escanear imágenes y lanzar el análisis dinámico), por lo que **no puede alojarse en plataformas serverless** como Vercel ni en planes gratuitos que suspenden el servicio.

Componente	Dónde se ejecuta	Motivo
Motor (orquestador)	Máquina local del equipo y runners de GitHub Actions	Necesita Docker; los runners de GitHub traen Docker preinstalado y son gratuitos en repositorios públicos.

Componente	Dónde se ejecuta	Motivo
Microservicios generados	Docker Compose (local o en el runner)	Entorno controlado, aislado y reproducible para pruebas y DAST.
PostgreSQL + pgvector	Contenedor en el mismo Docker Compose	Una línea de configuración, sin dependencia de servicios externos ni pausas por inactividad.
Dashboard Streamlit	Local junto al motor	Mismo proyecto Python, se levanta con un comando.
Dashboard Next.js (opcional)	Vercel	Solo la interfaz; consume la API del motor, que corre donde hay Docker.
API de LLM	Servicio en la nube del proveedor	Se accede por SDK con la clave en variables de entorno o GitHub Secrets.

4. Stack tecnológico completo

4.1 Resumen del stack

La tabla 4.1 resume todas las herramientas. Los capítulos siguientes justifican cada una en detalle.

Tabla 4.1: Stack tecnológico del framework.

Área	Herramienta	Función	Licencia
Núcleo de conocimiento			
Conocimiento	YAML en Git	Patrones, controles, mapeos OWASP/CWE, umbrales	–
Persistencia	PostgreSQL + pgvector	Base de conocimiento consultable, trazabilidad, búsqueda semántica	PostgreSQL
Razonamiento	Open Policy Agent (Rego)	Inferencia determinista y decisión de los gates	Apache 2.0
IA generativa	SDK de Gemini tras adaptador propio	Extracción, generación, corrección y explicación	Comercial (API)
Motor y backend			
Lenguaje	Python 3.12+	Lenguaje único del motor y de lo generado	PSF
API	FastAPI + Pydantic v2	API del motor y validación estricta de datos	MIT
CLI	Typer	Ejecutar el framework desde terminal y CI	MIT
Datos	SQLModel / SQLAlchemy + Alembic	Acceso a base de datos y migraciones	MIT
Gestión de dependencias	uv	Entornos y dependencias rápidos y reproducibles	MIT / Apache
Pipeline (fases 1–11)			
Fase 1	Docling	Leer PDF, DOCX y TXT de requisitos	MIT
Fase 2	Mermaid + ADRs	Diagramas C4 y registro de decisiones	MIT
Fase 3	Copier + Jinja2	Plantillas del microservicio	MIT
Fase 4	Semgrep CE / Opengrep	SAST con reglas propias	LGPL 2.1
Fase 4	Gitleaks	Detección de secretos	MIT
Fase 4	Trivy (fs, config)	SCA multi-ecosistema y configuraciones	Apache 2.0
Fase 4	Hadolint	Buenas prácticas en Dockerfile	GPL 3.0
Fases 5–6	Máquina de estados en Python	Refinamiento iterativo con salvaguardas	–
Fase 7	Traefik	API Gateway y service discovery	MIT
Fase 7	RabbitMQ	Event bus	MPL 2.0

Área	Herramienta	Función	Licencia
Fase 7	Keycloak	Identity Provider (OAuth2/OIDC)	Apache 2.0
Fase 8	Spectral + reglas OWASP API	Seguridad del contrato OpenAPI	Apache 2.0
Fase 8	Conftest (Rego)	Políticas de integración	Apache 2.0
Fase 9	pytest + pytest-cov	Pruebas unitarias y cobertura	MIT
Fase 9	Testcontainers	Pruebas de integración con servicios reales	MIT
Fase 9	Schemathesis	Pruebas de contrato y fuzzing de API	MIT
Fase 10	Docker + Trivy image	Construcción y escaneo de imágenes	Apache 2.0
Fase 10	Syft	SBOM en CycloneDX	Apache 2.0
Fase 11	OWASP ZAP (API scan)	DAST guiado por el contrato OpenAPI	Apache 2.0
Fase 11	Nuclei	DAST por plantillas (CVEs y configuraciones)	MIT
Infraestructura, interfaz y reportes			
CI/CD	GitHub Actions	Ejecutar el pipeline completo	Gratuito
Contenedores	Docker + Docker Compose	Entorno de ejecución y red de microservicios	Apache 2.0
Interfaz	Streamlit (o Next.js)	Dashboard de ejecución y métricas	Apache 2.0 / MIT
Reportes	Jinja2 + WeasyPrint	Informe de evidencia en HTML y PDF	BSD
Observabilidad	Logs JSON (structlog)	Registro estructurado del motor	MIT / Apache

4.2 Lenguajes de programación: dónde se usa cada uno

El proyecto usa lenguajes en tres lugares distintos. Es importante no confundirlos.

Parte	Lenguaje	Justificación
Motor del framework	Python	Todo el ecosistema necesario es nativo de Python: SDKs de LLM, Pydantic para salidas estructuradas, Docling, clientes de PostgreSQL y pgvector, generación de PDF. Es además el estándar académico en IA y seguridad.
Dashboard	Python (Streamlit) o TypeScript (Next.js)	Depende de la experiencia del equipo (sección 4.3).
Microservicios generados	Python (FastAPI)	Coherencia con el motor; FastAPI genera el contrato OpenAPI automáticamente, que es la entrada de Schemathesis, Spectral y ZAP.
Reglas y políticas	Rego, YAML	Lenguajes declarativos de OPA y Semgrep; son la forma computacional del conocimiento.

Plantillas	Jinja2	Motor de plantillas usado por Copier y por el generador de informes.
Pipeline	YAML (GitHub Actions)	Definición declarativa de los workflows de CI.

4.2.1 Por qué FastAPI y no otro framework

Alternativa	Ventaja	Por qué no se eligió
FastAPI (elegido)	OpenAPI automático, validación con Pydantic, asíncrono, muy rápido de aprender, excelente documentación.	–
Django REST Framework	Muy maduro, panel administrativo incluido.	Más pesado y con más código repetitivo; el OpenAPI requiere librerías adicionales.
Flask	Mínimo y simple.	Sin validación ni OpenAPI nativos; habría que ensamblar varias extensiones.
Spring Boot (Java)	Estándar empresarial en microservicios.	Segundo lenguaje, mayor verbosidad y más tiempo de arranque de contenedores.
NestJS (Node.js)	Arquitectura ordenada y TypeScript.	Segundo lenguaje distinto del motor.

4.3 Interfaz: Streamlit o Next.js

Criterio	Streamlit	Next.js
Lenguaje	Python, mismo proyecto que el motor	TypeScript, proyecto separado
Componentes listos	Carga de archivos, tablas, gráficos, barras de progreso	Hay que construirlos o usar librerías (shadcn/ui, Recharts)
Progreso en tiempo real	Básico (refresco periódico)	Completo (WebSockets o Server-Sent Events)
Aspecto	Prototipo profesional	Producto terminado
Despliegue	Junto al motor	Vercel, consumiendo la API del motor

5. Núcleo de conocimiento (capa superior)

5.1 Base de conocimiento

5.1.1 Qué contiene

Tipo de conocimiento	Contenido	Formato
Patrones de microservicios	API Gateway, Database per Service, Circuit Breaker, Saga, Event-Driven, BFF; con condiciones de aplicación.	YAML
Directrices Secure by Design	Mínimo privilegio, defensa en profundidad, validación de entrada, fallo seguro.	YAML
Controles de seguridad	Controles de OWASP ASVS asociados a requisitos y componentes.	YAML
Catálogos de vulnerabilidades	OWASP Top 10, OWASP API Security Top 10, CWE, referencias a CVE.	YAML + texto para RAG
Reglas arquitectónicas	Restricciones de diseño (p.ej., ningún servicio accede a la base de otro).	Rego
Reglas de código	Patrones inseguros y convenciones propias.	Reglas Semgrep
Políticas de aceptación	Umbral de cada gate (p.ej., cero hallazgos críticos o altos).	Rego
Dependencias permitidas	Lista de paquetes aprobados para el código generado.	YAML
Decisiones validadas	Decisiones de ejecuciones anteriores con su resultado.	PostgreSQL

5.1.2 Cómo se almacena

El conocimiento se escribe y versiona en Git y se sincroniza a PostgreSQL al iniciar el motor. Los documentos de referencia (descripciones de CWE, guías OWASP) se dividen en fragmentos y se indexan con embeddings en **pgvector** para recuperación semántica (RAG).

Listing 5.1: Ejemplo de patrón en la base de conocimiento (knowledge_base/patrones/api_gateway.yaml).

```
id: arq.gateway.001
patron: api_gateway
descripcion: Punto único de entrada para los clientes
aplica_cuando:
  - etiqueta: punto_unico_acceso
  - etiqueta: autenticacion_centralizada
  - numero_servicios_minimo: 2
controles_asociados: [ASVS-V4.1.1, ASVS-V13.1.3]
cwe_mitigadas: [CWE-284, CWE-306]
implementacion: traefik
```

5.1.3 Por qué PostgreSQL + pgvector

Alternativa	Ventaja	Por qué no se eligió
PostgreSQL + pgvector (elegido)	Una sola base para datos relacionales, trazabilidad y vectores; SQL estándar; contenedor oficial.	—
Neo4j (grafo)	Modela relaciones de trazabilidad de forma natural.	Segunda base de datos, lenguaje Cypher adicional; PostgreSQL con claves foráneas cubre la necesidad.
Chroma / Qdrant	Especializadas en vectores.	Base adicional solo para RAG; pgvector basta para este volumen.
Supabase	PostgreSQL gestionado con APIs listas.	Pausa proyectos inactivos en plan gratuito y saca datos de seguridad a un servicio externo; queda como opción para un dashboard público.
Ontología OWL/Protégé	Máxima formalidad semántica.	Curva de aprendizaje alta y difícil de integrar en CI.

5.2 Motor de razonamiento

El motor combina dos componentes con responsabilidades separadas:

- **Open Policy Agent (OPA)**: razonamiento determinista. Recibe un JSON (modelo de requisitos, arquitectura propuesta, hallazgos SARIF, configuración) y evalúa políticas Rego que devuelven decisiones con su justificación.
- **LLM**: razonamiento generativo. Produce propuestas, código, correcciones y explicaciones en lenguaje natural, siempre validadas después por OPA y por las herramientas.

Listing 5.2: Inferencia de patrones con justificación (policies/arquitectura/patrones.rego).

```
package arquitectura.patrones

import rego.v1

recomendaciones contains rec if {
  some r in input.requisitos
  "punto_unico_acceso" in r.etiquetas
  count(input.servicios) >= 2
  rec := {
    "patron": "api_gateway",
    "regla": "arq.gateway.001",
    "justificacion": sprintf("Se aplica API Gateway porque %s exige un punto único de acceso", [r.id]),
  }
}
```

Listing 5.3: Política de aceptación del Security Gate 1 (policies/gates/gate1.rego).

```
package gates.gate1

import rego.v1

default aprobado := false

aprobado if count(bloqueantes) == 0

bloqueantes contains h if {
```

```

    some h in input.hallazgos
    h.severidad in {"critica", "alta"}
}

bloqueantes contains h if {
    some h in input.hallazgos
    h.herramienta == "gitLeaks"
}

```

5.2.1 Por qué OPA

Alternativa	Ventaja	Por qué no se eligió
OPA / Rego (elegido)	Estándar CNCF graduado; el mismo motor sirve para inferencia de patrones, gates y políticas de configuración (vía Conftest); decisiones explicables y testeables con <code>opa test</code> .	—
Motor de reglas en Python (experta, <code> durable_rules </code>)	Mismo lenguaje que el motor.	Proyectos con poco mantenimiento; no reutilizables para validar configuraciones.
Prolog / Clingo	Máxima expresividad lógica.	Difícil de integrar en CI y de mantener para el equipo.
Solo LLM	Flexible, sin escribir reglas.	No determinista ni auditable; incompatible con el principio P1.

5.3 Trazabilidad y retroalimentación

Cada ejecución registra la cadena completa desde el requisito hasta la evidencia. El modelo de datos de la figura 5.1 permite generar el informe final y las métricas mediante consultas SQL.

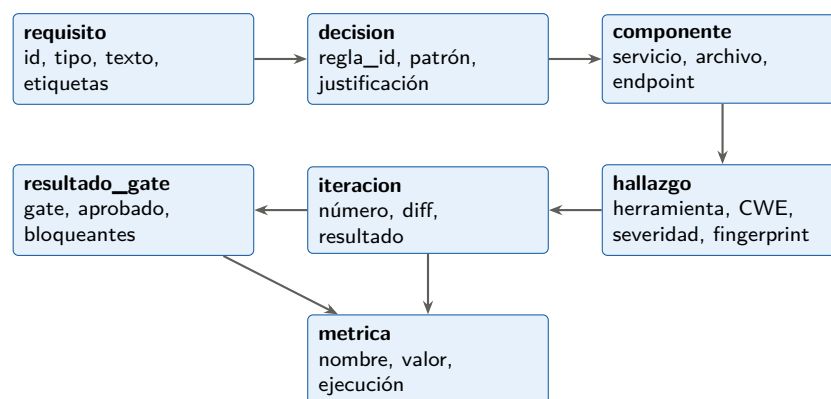


Figura 5.1: Modelo de datos de trazabilidad (simplificado). Todas las tablas incluyen `ejecucion_id` y la versión del conocimiento usada.

La **retroalimentación** cierra el ciclo: al final de cada ejecución, las correcciones que resolvieron un hallazgo sin romper pruebas se guardan como *decisiones validadas*. Si una misma corrección se repite, se propone al equipo convertirla en una regla nueva de la base de conocimiento, que se revisa por pull request antes de incorporarse (aprendizaje supervisado por humanos).

5.4 SARIF como formato común

SARIF (Static Analysis Results Interchange Format) es un estándar OASIS para resultados de análisis. La mayoría de las herramientas elegidas lo generan de forma nativa (Sengrep, Gitleaks, Trivy, Hadolint, Nuclei y Spectral); para las demás (Schemathesis y, según la versión, ZAP) se escribe un conversor pequeño a partir de su salida JSON. Ventajas:

- Un solo parser en el motor para todas las herramientas.
- Los *fingerprints* identifican el mismo hallazgo entre iteraciones, lo que permite medir qué se corrigió y detectar regresiones.
- GitHub muestra los archivos SARIF en la pestaña de seguridad del repositorio sin trabajo adicional.

6. Uso de modelos de lenguaje (LLM)

6.1 Principio de uso

Los LLM se usan únicamente en tareas donde el lenguaje natural o la generación de código aportan valor, y siempre con tres condiciones:

1. **Salida estructurada:** toda respuesta se valida contra un esquema Pydantic. Si no cumple, se reintenta; nunca se procesa texto libre.
2. **Contexto desde la base de conocimiento:** el prompt incluye las reglas, controles y fragmentos OWASP/CWE relevantes recuperados por RAG, no el conocimiento genérico del modelo.
3. **Verificación posterior:** lo que produce el LLM pasa siempre por OPA, por las herramientas de análisis o por las pruebas. El LLM nunca tiene la última palabra.

6.2 Mapa de uso por fase

Tabla 6.1: Dónde y para qué se usa cada modelo.

Fase	Tarea del LLM	Modelo	Verificación
1	Extraer requisitos del texto, clasificar RF/RNF, identificar actores, entidades y etiquetas de seguridad	Rápido (familia Gemini Flash)	Esquema Pydantic Requisito ; revisión opcional en el dashboard
2	Proponer bounded contexts y división en microservicios	Razonamiento (familia Gemini Pro)	OPA valida reglas arquitectónicas e infiere patrones
2	Redactar el texto de los ADR a partir de las decisiones de OPA	Rápido	La decisión ya la tomó OPA; el LLM solo redacta
3	Completar la lógica de negocio dentro de las plantillas	Razonamiento	Semgrep, pruebas unitarias, lista de dependencias permitidas
3	Generar pruebas unitarias iniciales	Razonamiento	Ejecución con pytest
5–6	Proponer corrección de un hallazgo SAST/SCA	Razonamiento	Reescaneo, pruebas y control de regresiones
5–6	Explicar el hallazgo y la corrección para el informe	Rápido	Contenido informativo, no decisorio

Fase	Tarea del LLM	Modelo	Verificación
Transversal	Embeddings de OWASP, CWE y guías para RAG	Modelo de embeddings	Evaluación de recuperación con consultas de prueba
11	Resumen ejecutivo del informe final	Rápido	Datos numéricos provienen de la base, no del LLM

Dónde NO se usa el LLM

Aprobar o rechazar un Security Gate, asignar severidades, calcular métricas, decidir si una prueba pasó y seleccionar el patrón arquitectónico final. Estas decisiones son deterministas y se toman con OPA, las herramientas y consultas a la base de datos.

6.3 Selección de modelos

- **Proveedor principal: Gemini o llm a definir**, por su plan gratuito generoso para desarrollo, soporte de salida estructurada con esquemas JSON y ventana de contexto amplia, útil para enviar archivos de código completos.
- **Dos niveles de modelo**: uno rápido y económico para extracción, clasificación y redacción, y uno de razonamiento para arquitectura, generación y corrección de código. Las versiones concretas cambian con frecuencia; al iniciar el proyecto se fija la versión vigente en la documentación del proveedor y se registra en la configuración, para que los experimentos sean reproducibles.
- **Embeddings**: el modelo de embeddings del mismo proveedor, o un modelo abierto multilingüe ejecutado localmente (por ejemplo, de la familia multilingual-e5) si se quiere evitar costes y dependencia externa.
- **Temperatura baja** (0 a 0,2) en extracción y corrección para reducir la variabilidad; se documenta en cada experimento.

6.4 Adaptador de proveedor

El motor no llama al SDK directamente desde las fases, sino a través de una interfaz propia. Esto permite cambiar de proveedor con un parámetro de configuración y, sobre todo, **comparar modelos en la evaluación DSR** (por ejemplo, Gemini frente a Claude, GPT o un modelo local con Ollama), lo que enriquece los resultados de la tesis.

Listing 6.1: Interfaz del adaptador de LLM (orchestrator/llm/base.py).

```
from typing import Protocol, TypeVar
from pydantic import BaseModel

T = TypeVar("T", bound=BaseModel)

class ProveedorLLM(Protocol):
    def generar_estructurado(
        self, prompt: str, esquema: type[T], nivel: str = "rapido"
    ) -> T:
        """Devuelve una instancia validada del esquema o lanza error."""
        ...
```

Alternativa	Ventaja	Por qué no se eligió
SDK oficial + adaptador propio (elegido)	Control total, fácil de depurar, sin dependencias pesadas.	–
LangChain / LangGraph	Muchos componentes prefabricados, grafos de agentes.	Capa de abstracción grande que dificulta depurar; la lógica de este proyecto es un flujo conocido que no necesita agentes autónomos.
LiteLLM	Interfaz unificada para muchos proveedores.	Opción válida si se quieren comparar muchos modelos; se puede usar dentro del adaptador.

6.5 Riesgos específicos del uso de LLM

Riesgo	Mitigación
Prompt injection desde el documento de requisitos	El documento se trata como dato, delimitado en el prompt; la salida se valida con Pydantic; el LLM no tiene acceso a herramientas ni a ejecución.
Paquetes alucinados (<i>slopsquatting</i>)	Toda dependencia generada se valida contra la lista permitida de la base de conocimiento y pasa por SCA.
Corrección que elimina funcionalidad	Salvaguardas del motor de refinamiento (sección 7.5).
Variabilidad entre ejecuciones	Temperatura baja, versión de modelo fijada, varias repeticiones por experimento y registro de prompts y respuestas.
Fuga de datos sensibles	No se envían secretos ni datos reales; las claves se gestionan en variables de entorno y GitHub Secrets.
Coste y límites de uso	Caché de respuestas por hash del prompt durante el desarrollo; modelo rápido por defecto.

7. Pipeline de construcción y validación (capa media)

Cada fase se describe con el mismo formato: objetivo, herramientas, entradas, salidas y justificación frente a alternativas.

7.1 Fase 1: Adquisición de requisitos

Objetivo: convertir documentos en lenguaje natural en un modelo estructurado de requisitos.

Paso	Herramienta y uso
Captura	Carga en el dashboard, por CLI (<code>framework run requisitos.pdf</code>) o subiendo el archivo a <code>requirements/</code> en el repositorio.
Extracción	Docling convierte PDF y DOCX a texto estructurado conservando títulos, listas y tablas.
Normalización	Limpieza en Python (espacios, numeración, división en unidades de requisito).
Análisis semántico y clasificación	LLM rápido con esquema Pydantic: tipo (RF/RNF), actor, acción, entidades, servicio sugerido y etiquetas de seguridad.
Validación	Pydantic rechaza salidas incompletas; el dashboard permite revisar y corregir antes de continuar.

Entrada: PDF, DOCX, TXT o historias de usuario. **Salida:** `modelo_requisitos.json`.

Listing 7.1: Ejemplo de requisito estructurado.

```
{
  "id": "RF-01",
  "tipo": "funcional",
  "actor": "cliente",
  "accion": "registrarse con correo",
  "entidades": ["Usuario"],
  "servicio_sugerido": "usuarios",
  "etiquetas": ["autenticacion", "validacion_entrada", "hash_password"]
}
```

Por qué Docling: proyecto abierto de IBM que maneja bien la estructura de documentos (tablas, títulos, listas) y exporta a Markdown o JSON. Frente a *PyPDF* solo extrae texto plano; frente a *Unstructured* es más ligero de instalar para este caso de uso.

7.2 Fase 2: Análisis y propuesta arquitectónica

Objetivo: decidir qué microservicios crear, qué patrones aplicar y por qué.

1. El LLM de razonamiento propone bounded contexts y la división en servicios a partir del modelo de requisitos.
2. OPA evalúa la propuesta contra las reglas arquitectónicas (p. ej., cada servicio con su propia base de datos, sin dependencias cíclicas) e infiere los patrones y controles aplicables, con su justificación.

3. Si OPA rechaza la propuesta, el motivo se devuelve al LLM para una nueva propuesta (máximo tres intentos).
4. Se generan los ADR (Architecture Decision Records) en Markdown y el diagrama C4 en Mermaid.

Salida: `arquitectura.json`, `docs/adr/*.md`, `docs/arquitectura.md`.

Por qué ADR + Mermaid: los ADR son el formato estándar de la industria para documentar decisiones con contexto y consecuencias, y encajan con la trazabilidad. Mermaid se escribe como texto, se versiona en Git y GitHub lo renderiza sin herramientas adicionales. Structurizr es más completo para C4 pero añade otra herramienta y otro lenguaje.

7.3 Fase 3: Generación de la estructura interna

Objetivo: producir el código de cada microservicio.

- **Copier** genera el esqueleto determinista desde una plantilla: entidades, DTO, repositorios, servicios, controladores, endpoints, configuración, Dockerfile y pruebas base.
- La plantilla ya incluye los controles transversales: autenticación JWT validada contra Keycloak, validación con Pydantic, logging estructurado, manejo seguro de errores y configuración por variables de entorno.
- El LLM solo completa la lógica de negocio en puntos marcados de la plantilla.

Alternativa	Ventaja	Por qué no se eligió
Copier (elegido)	Plantillas Jinja2, preguntas tipadas y capacidad de <i>actualizar</i> proyectos ya generados cuando cambia la plantilla.	–
Cookiecutter	Más conocido y muy simple.	No actualiza proyectos existentes; válido si el equipo ya lo conoce.
Generación 100 % por LLM	Máxima flexibilidad.	Resultados inconsistentes y difíciles de validar; más vulnerabilidades.

Decisión clave

La combinación *plantilla determinista + LLM acotado* reduce drásticamente la superficie de error y permite medir en la evaluación cuánto aporta cada parte.

7.4 Fase 4: Security Gate 1, análisis estático

Herramienta	Qué analiza	Ventaja frente a alternativas
Semgrep CE u Opengrep	SAST: inyecciones, criptografía débil, deserialización insegura y reglas propias del proyecto	Muy rápido; reglas en YAML legibles que forman parte de la base de conocimiento. SonarQube requiere servidor y base de datos propios; Bandit solo cubre Python y no admite reglas propias con la misma facilidad.

Gitleaks	Secretos (claves, tokens, contraseñas)	Binario único, milisegundos por escaneo, reglas configurables. Más ligero que TruffleHog para este uso.
Trivy fs	SCA: vulnerabilidades en dependencias de cualquier ecosistema	Cubre Python y otros lenguajes con la misma herramienta que se usa en la fase 10; pip-audit solo cubre Python.
Trivy config	Configuraciones inseguras en Dockerfile y Compose	Misma herramienta, sin instalar Checkov por separado.
Hadolint	Buenas prácticas en Dockerfile	Estándar de facto, salida SARIF, muy rápido.

Nota de licencia de Sengrep

En diciembre de 2024 Sengrep cambió la licencia de sus reglas oficiales y trasladó funciones a su edición comercial; la comunidad respondió con el fork **Opengrep**, totalmente abierto. Sengrep CE sigue siendo válido para uso académico. En ambos casos se recomienda escribir y usar reglas propias del proyecto, que además forman parte de la base de conocimiento.

Decisión: todos los resultados se convierten a SARIF, se envían a OPA con la política `gates.gate1` y se registran en la base de trazabilidad. Si hay bloqueantes, el flujo pasa a las fases 5–6.

7.5 Fases 5 y 6: Refinamiento iterativo guiado por conocimiento

Objetivo: corregir los hallazgos del gate sin romper la funcionalidad, dejando registro de cada paso.

El motor de refinamiento es una **máquina de estados explícita en Python**. Recibe los hallazgos, la arquitectura, el código y la base de conocimiento, y sigue los pasos del diagrama del proyecto: identificar, explicar, priorizar, seleccionar control y refinar.

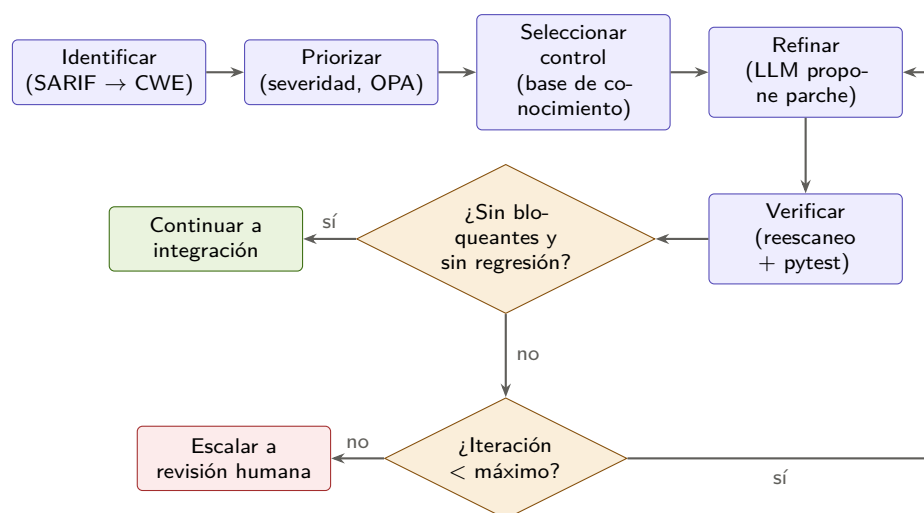


Figura 7.1: Máquina de estados del motor de refinamiento.

7.5.1 Salvaguardas obligatorias

1. **Máximo de iteraciones** configurable (por defecto 5). Si no converge, se escala a revisión humana y queda registrado.
2. **Pruebas tras cada cambio:** si una prueba unitaria que antes pasaba ahora falla, el parche se rechaza.
3. **Control de regresión por fingerprint SARIF:** un parche que introduce hallazgos nuevos se rechaza.
4. **Integridad funcional:** se rechazan parches que eliminan funciones, endpoints o pruebas (verificación por análisis del árbol sintáctico con el módulo `ast` de Python).
5. **Alcance acotado:** el parche solo puede modificar los archivos implicados en el hallazgo.
6. **Registro completo:** cada iteración guarda el diff, el prompt, la respuesta y el resultado en la base de trazabilidad.

Qué puede modificar: código, configuración, dependencias (solo hacia versiones de la lista permitida), contratos de API y, en casos justificados, decisiones arquitectónicas (lo que devuelve el flujo a la fase 2).

Por qué una máquina de estados propia y no LangGraph: el flujo es conocido y finito; una implementación de menos de 300 líneas en Python es más fácil de depurar, probar y explicar en la tesis que un grafo de agentes con una librería externa.

7.6 Fase 7: Integración de microservicios

Componente	Herramienta	Justificación y alternativas
API Gateway	Traefik	Se configura con etiquetas en Docker Compose, detecta servicios automáticamente, incluye TLS, rate limiting y middleware de autenticación. Kong y NGINX requieren más configuración manual; Spring Cloud Gateway obliga a usar Java.
Service Discovery	Traefik (proveedor Docker)	El descubrimiento por etiquetas cubre la necesidad sin instalar Consul o Eureka.
Load Balancer	Traefik	Balanceo integrado entre réplicas de un servicio.
Event Bus	RabbitMQ	Muy documentado, con interfaz web útil para demostraciones y librerías maduras en Python. NATS es más ligero; Kafka es excesivo para este alcance.
Identity Provider	Keycloak (modo desarrollo)	Estándar abierto de OAuth2/OIDC con consola de administración; se importa la configuración del realm desde un archivo JSON. Auth0 es comercial; implementar un servicio de autenticación propio añade riesgos.
Observabilidad	Logs JSON + cabecera de correlación	Suficiente para trazar peticiones entre servicios. OpenTelemetry con Grafana queda como mejora opcional.
Service Mesh	No se implementa	Fuera de alcance; se justifica como trabajo futuro (mTLS entre servicios).

Listing 7.2: Fragmento de docker-compose.yml generado.

```

services:
  gateway:
    image: traefik:v3.1
    command:
      - --providers.docker=true
      - --providers.docker.exposedbydefault=false
      - --entrypoints.web.address=:80
    ports: ["80:80"]
    volumes: ["/var/run/docker.sock:/var/run/docker.sock:ro"]

  usuarios:
    build: ./services/usuarios
    read_only: true
    user: "10001"
    labels:
      - traefik.enable=true
      - traefik.http.routers.usuarios.rule=PathPrefix('/api/usuarios')
      - traefik.http.middlewares.ratelimit.ratelimit.average=50
      - traefik.http.routers.usuarios.middlewares=ratelimit@docker
    env_file: ./services/usuarios/.env

```

7.7 Fase 8: Security Gate 2, seguridad de integración

Herramienta	Qué valida	Ventaja
Spectral con reglas OWASP API Security	Contratos OpenAPI: autenticación declarada en cada endpoint, sin HTTP plano, límites en cadenas y arreglos, respuestas de error definidas.	Valida la seguridad <i>antes</i> de levantar los servicios; las reglas son YAML.
Conftest (Rego)	Docker Compose y configuración de Traefik: TLS, middleware de autenticación, contenedores sin privilegios, sistema de archivos de solo lectura, secretos fuera de variables en claro, redes separadas.	Reutiliza el mismo lenguaje Rego y la misma base de conocimiento que OPA.
OPA (gates.gate2)	Decisión final del gate a partir de ambos resultados.	Decisión explicable y registrada.

7.8 Fase 9: Pruebas automatizadas

Tipo de prueba	Herramienta	Uso y ventaja
Unitarias	pytest + pytest-cov	Estándar de Python; la plantilla trae pruebas base y el LLM genera pruebas adicionales. Mide cobertura.
Integración	Testcontainers	Levanta PostgreSQL y RabbitMQ reales dentro de la prueba; evita falsos positivos de los mocks.
Funcionales / end-to-end	pytest + httpx	Recorren flujos completos a través del gateway (registro, login, compra).

Contrato y fuzzing	Schemathesis	Lee el OpenAPI y genera cientos de casos, incluidos datos inválidos, verificando que las respuestas cumplan el contrato y que no haya errores 500. Pact es más completo para contratos consumidor-proveedor pero requiere mantener contratos manuales.
Regresión	Suite completa de pytest	Se ejecuta en cada iteración y al final.
Seguridad	pytest + casos derivados de la base de conocimiento	Pruebas de autorización (acceso a recursos ajenos, tokens inválidos o expirados).

Criterio de paso: todas las pruebas en verde, cobertura mínima configurable (por ejemplo, 70 %) y cero errores 500 en Schemathesis, evaluado por OPA.

7.9 Fase 10: Contenerización

- **Docker:** imágenes multi-etapa con base `python:3.12-slim`, usuario no root y sin herramientas de compilación en la imagen final.
- **Trivy image:** escaneo de CVEs del sistema operativo y de las librerías de la imagen. Frente a Gype es equivalente en calidad, pero Trivy ya se usa en la fase 4, lo que reduce herramientas.
- **Syft:** genera el SBOM en CycloneDX, que se adjunta al informe como inventario verificable de componentes.
- Variables de entorno y secretos: archivos `.env` excluidos de Git y secretos de Docker Compose; en CI, GitHub Secrets.

7.10 Fase 11: DAST y Security Gate 3

Los servicios se despliegan con Docker Compose en un entorno controlado (red aislada del runner) y se atacan de forma automática.

Herramienta	Qué aporta	Por qué ambas
OWASP ZAP (<code>zap-api-scan.py</code>)	Importa el OpenAPI y ejecuta un escaneo activo contra cada endpoint: inyección, fallos de autenticación, cabeceras, exposición de datos.	Es el DAST de referencia de OWASP y el único de los dos que prueba activamente la API a partir del contrato. Se limita a 5–10 minutos por servicio para evitar timeouts en CI.
Nuclei	Plantillas YAML para CVEs conocidas, configuraciones erróneas y paneles expuestos.	Muy rápido y ligero; complementa a ZAP con detección basada en firmas. No sustituye a ZAP porque no rastrea ni ataca la lógica de la API.
Pruebas autenticadas	Token de Keycloak inyectado en ZAP y Schemathesis	Permite probar endpoints protegidos, no solo los públicos.

Gate 3: OPA evalúa los resultados DAST junto con los de Trivy image y la fase 9. Si se aprueba, se genera el entregable final; si no, el hallazgo vuelve al motor de refinamiento.

8. Infraestructura y CI/CD

8.1 GitHub Actions como orquestador del pipeline

GitHub Actions ejecuta el framework completo en cada cambio de requisitos o de conocimiento. Ventajas frente a las alternativas:

- Frente a **Jenkins**: no requiere servidor propio ni mantenimiento.
- Frente a **GitLab CI**: equivalente en funciones; se elige GitHub por la integración nativa con SARIF en la pestaña de seguridad y por ser gratuito en repositorios públicos.
- Los runners traen Docker preinstalado, requisito del pipeline.
- Con la herramienta **act** se pueden ejecutar los workflows localmente para depurar.

Listing 8.1: Workflow principal (.github/workflows/framework.yml), simplificado.

```
name: framework-desarrollo-seguro
on:
  push:
    paths: ["requirements/**", "knowledge_base/**", "policies/**"]
    workflow_dispatch:
jobs:
  pipeline:
    runs-on: ubuntu-latest
    timeout-minutes: 60
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env: { POSTGRES_PASSWORD: ${ secrets.DB_PASSWORD } }
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - name: Instalar herramientas de seguridad
        run: ./scripts/instalar_herramientas.sh
      - name: Probar políticas OPA
        run: opa test policies/ -v
      - name: Ejecutar framework
        env:
          GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
        run: uv run framework run requirements/ --salida salida/
      - name: Publicar SARIF
        uses: github/codeql-action/upload-sarif@v3
        with: { sarif_file: salida/security/sarif/ }
      - uses: actions/upload-artifact@v4
        with: { name: entregable, path: salida/ }
```

8.2 Entornos

Entorno	Uso
Local	Desarrollo del framework: Docker Desktop o Docker Engine, uv, herramientas instaladas por script. Recomendado 16 GB de RAM (Keycloak, RabbitMQ, PostgreSQL y los microservicios a la vez).

CI (GitHub Actions)	Ejecuciones reproducibles para la evaluación y demostración del DevSecOps continuo.
Entorno controlado de pruebas	Red Docker aislada donde se despliegan los microservicios generados para las fases 9 y 11.

9. Entradas y salidas del framework

9.1 Formas de entrada

1. **Dashboard:** se arrastra el documento, se eligen opciones (nivel de seguridad, iteraciones máximas) y se ejecuta. Ideal para demostraciones.
2. **CLI:** `framework run requisitos.pdf -nivel alto`. Ideal para experimentos por lotes.
3. **Repositorio:** subir el documento a `requirements/` dispara el workflow. Demuestra el DevSecOps continuo.

9.2 Salida: el entregable

Listing 9.1: Estructura del entregable generado.

```
salida_proyecto/
|-- services/
|   |-- usuarios/           # FastAPI: entidades, DTO, repositorios, controladores, tests
|   |-- catalogo/
|   |-- pedidos/
|   '-- pagos/
|-- gateway/               # configuración de Traefik
|-- keycloak/              # realm exportado
|-- docker-compose.yml     # levanta todo el sistema con un comando
|-- openapi/               # contrato de cada microservicio
|-- docs/
|   |-- adr/               # decisiones arquitectónicas justificadas
|   '-- arquitectura.md    # diagrama C4 en Mermaid
|-- security/
|   |-- sarif/             # resultados de todas las herramientas
|   '-- sbom.json          # inventario de componentes (CycloneDX)
|-- metricas.json
'-- informe_evidencia.pdf
```

9.3 Informe de evidencia y trazabilidad

Generado con **Jinja2** (plantilla HTML) y **WeasyPrint** (conversión a PDF). Contiene:

- **Matriz de trazabilidad:** requisito → decisión → microservicio → archivos → pruebas → controles.
- **Decisiones arquitectónicas** con la regla de la base de conocimiento que las justifica.
- **Historial de refinamiento:** hallazgos, iteraciones y correcciones aplicadas.
- **Métricas de seguridad, calidad, arquitectura, eficiencia y efectividad** (capítulo 11).
- **Veredicto de cada Security Gate** con los bloqueantes resueltos.
- **Resumen ejecutivo** redactado por el LLM a partir de datos de la base (los números nunca los genera el LLM).

10. Seguridad del propio framework

Un framework de seguridad debe aplicarse a sí mismo sus principios.

Riesgo	Control
Código generado malicioso o defectuoso	Solo se ejecuta dentro de contenedores con red aislada, usuario no root y límites de recursos.
Secretos del framework	Claves en GitHub Secrets y <code>.env</code> fuera de Git; Gitleaks también escanea el repositorio del framework.
Dependencias del framework	Trivy y Dependabot sobre el propio repositorio.
Manipulación del conocimiento	Cambios en <code>knowledge_base/</code> y <code>policies/</code> solo por pull request revisado; <code>opa test</code> obligatorio en CI.
Prompt injection	Ver capítulo de LLM.
Socket de Docker expuesto a Traefik	Montado en solo lectura; en entornos compartidos, uso de un proxy de socket.

11. Evaluación del artefacto (ciclo DSR)

11.1 Relación entre el ciclo DSR y el plan

Etapas DSR	Qué se hace en este proyecto
Problema / necesidad	Revisiones de seguridad manuales, tardías y aisladas en microservicios; falta de trazabilidad entre requisitos y controles.
Diseño del artefacto	Arquitectura de tres capas, principios P1–P6 y este plan.
Construcción	Implementación por sprints (capítulo ??).
Evaluación	Experimentos con conjuntos de requisitos y comparación con líneas base.
Refinamiento	Ajuste de reglas, plantillas y prompts según los resultados.
Reevaluación	Repetición de los experimentos con el conocimiento actualizado.
Artefacto validado	Framework consolidado y documentado.
Actualización del conocimiento	Nuevas reglas y lecciones incorporadas a la base de conocimiento.

11.2 Métricas

Dimensión	Métrica	Fuente
Seguridad	Número de vulnerabilidades por severidad; riesgo residual; porcentaje reducido entre el primer gate y el final	SARIF + base de trazabilidad
Calidad	Cobertura de pruebas; porcentaje de requisitos cubiertos; errores funcionales	pytest-cov, Schemathesis, matriz de trazabilidad
Arquitectura	Porcentaje de requisitos trazados; cumplimiento de reglas arquitectónicas; acoplamiento entre servicios	OPA, grafo de dependencias
Eficiencia	Iteraciones de refinamiento; tiempo total y tiempo hasta cada gate; tokens consumidos	Registros del motor
Efectividad	Vulnerabilidades iniciales frente a finales; porcentaje de controles de seguridad satisfechos	Base de trazabilidad

11.3 Diseño experimental

11.3.1 Conjuntos de requisitos

Se preparan entre tres y cinco documentos de requisitos de dominios distintos (por ejemplo, tienda en línea, gestión académica, citas médicas), con complejidad creciente. Cada uno se ejecuta varias veces para medir la variabilidad.

11.3.2 Líneas base de comparación

Configuración	Descripción
B0: solo LLM	El LLM genera los microservicios desde los requisitos, sin plantillas, sin base de conocimiento ni gates.
B1: LLM + gates	Igual que B0 pero con los Security Gates y refinamiento, sin base de conocimiento ni RAG.
F: framework completo	Plantillas, base de conocimiento, OPA, RAG, gates y refinamiento.

La diferencia entre B0, B1 y F permite atribuir la mejora a cada componente, que es el argumento central de la tesis. Opcionalmente se repite F con distintos modelos gracias al adaptador de proveedor.

11.3.3 Calibración de las herramientas dinámicas

Para demostrar que el DAST detecta lo que debe, se ejecuta la fase 11 contra aplicaciones deliberadamente vulnerables con vulnerabilidades conocidas:

- **VAmPI**: API REST vulnerable basada en Flask, con vulnerabilidades del OWASP API Top 10.
- **OWASP crAPI**: aplicación de microservicios vulnerable, muy cercana a la arquitectura del proyecto.

Se mide la tasa de detección frente a las vulnerabilidades documentadas de cada aplicación.

11.4 Dashboard de métricas

El dashboard (Streamlit o Next.js) muestra por ejecución: progreso por fase, decisiones tomadas con su justificación, hallazgos por gate e iteración, comparación entre configuraciones B0, B1 y F, y descarga del entregable.

12. Estructura del repositorio del framework

```
framework/
|-- knowledge_base/          # YAML: patrones, controles, mapeos CWE/OWASP, umbrales
|   |-- patrones/
|   |-- controles/
|   |-- vulnerabilidades/
|   '-- dependencias_permitidas.yaml
|-- policias/                # Rego
|   |-- arquitectura/        # inferencia de patrones y reglas de diseño
|   |-- gates/               # gate1.rego, gate2.rego, gate3.rego
|   '-- integracion/         # políticas para Conftest
|-- semgrep_rules/           # reglas SAST propias
|-- spectral/                # reglas de contrato OpenAPI
|-- templates/
|   '-- microservicio_fastapi/ # plantilla Copier
|-- orchestrator/
|   |-- cli.py                # Typer
|   |-- api.py                # FastAPI
|   |-- llm/                  # adaptador de proveedores y prompts
|   |-- ingest/               # fase 1
|   |-- architecture/         # fase 2
|   |-- generation/           # fase 3
|   |-- gates/                # ejecución de herramientas y conversión a SARIF
|   |-- refinement/           # máquina de estados
|   |-- integration/          # fases 7-8
|   |-- testing/              # fase 9
|   |-- packaging/            # fases 10-11
|   '-- report/               # informe Jinja2 + WeasyPrint
|-- traceability/            # modelos SQLAlchemy y migraciones Alembic
|-- dashboard/               # Streamlit (o carpeta web/ con Next.js)
|-- evaluation/
|   |-- datasets/             # documentos de requisitos de prueba
|   |-- baselines/            # configuraciones B0, B1, F
|   '-- experimentos.py
|-- scripts/instalar_herramientas.sh
|-- docker-compose.dev.yml    # PostgreSQL + pgvector, Keycloak, RabbitMQ
|-- pyproject.toml
'-- .github/workflows/
```

13. Riesgos del proyecto y mitigación

Riesgo	Impacto	Mitigación
Alcance excesivo	Alto	Un solo lenguaje, sin Kubernetes ni service mesh; walking skeleton primero.
Resultados del LLM inconsistentes	Alto	Plantillas deterministas, salida estructurada, temperatura baja, versión fijada.
Tiempos de CI demasiado largos	Medio	Límite de tiempo en ZAP, caché de imágenes y dependencias, ejecución paralela de escáneres.
Límites o costes de la API de LLM	Medio	Caché de respuestas, modelo rápido por defecto, opción de modelo local con Ollama.
Falta de recursos en equipos locales	Medio	Ejecutar la suite completa en GitHub Actions; Keycloak y RabbitMQ solo en fases 7 a 11.
Curva de aprendizaje de Rego	Medio	Empezar con políticas simples y usar el Rego Playground de OPA para probar.
Cambios de licencia de herramientas	Bajo	Alternativas abiertas identificadas (Opengrep para Semgrep).

A. Glosario

ADR

Architecture Decision Record: documento breve que registra una decisión arquitectónica, su contexto y sus consecuencias.

DAST

Dynamic Application Security Testing: análisis de seguridad de una aplicación en ejecución.

DSR

Design Science Research: metodología de investigación orientada a construir y evaluar artefactos.

KBS

Knowledge-Based System: sistema que razona a partir de una base de conocimiento explícita.

OPA

Open Policy Agent: motor de políticas de propósito general que evalúa reglas escritas en Rego.

RAG

Retrieval-Augmented Generation: técnica que añade al prompt información recuperada de una base de conocimiento.

SARIF

Static Analysis Results Interchange Format: estándar OASIS para resultados de análisis.

SAST

Static Application Security Testing: análisis de seguridad del código fuente sin ejecutarlo.

SBOM

Software Bill of Materials: inventario de componentes y dependencias de un software.

SCA

Software Composition Analysis: análisis de vulnerabilidades en dependencias de terceros.

Walking skeleton

Implementación mínima que recorre todo el sistema de punta a punta.